

Execution of natural language requirements using State Machines synthesised from Behavior Trees

Soon-Kyeong Kim^{a,*}, Toby Myers^b, Marc-Florian Wendland^c, Peter A. Lindsay^{d,a}

^a Software Systems Research Group, Queensland Research Lab, National ICT, Australia

^b Griffith University, Nathan, QLD 4111, Australia

^c Fraunhofer Institut FOKUS, Berlin, Germany

^d School of ITEE, The University of Queensland, QLD 4072, Australia

ARTICLE INFO

Article history:

Received 18 November 2011

Received in revised form 5 April 2012

Accepted 5 June 2012

Available online 19 June 2012

Keywords:

Requirements

Requirements Validation

Behavior Trees

Behavior Engineering

MDE

Model Transformation

UML State Machine

ABSTRACT

This paper defines a transformation from Behavior Tree models to UML state machines. Behavior Trees are a graphical modelling notation for capturing and formalising dynamic system behaviour described in natural language requirements. But state machines are more widely used in software development, and are required for use with many tools, such as test case generators. Combining the two approaches provides a formal path from natural language requirements to an executable model of the system. This in turn facilitates requirements validation and transition to model-driven software development methods. The approach is demonstrated by defining a mapping from Behavior Trees to UML state machines using the ATLAS Transformation Language (ATL) in the Eclipse Modeling Framework. A security-alarm system case study is used to illustrate the use of Behavior Trees and execution to debug requirements.

© 2012 Elsevier Inc. All rights reserved.

1. Introduction

Behavior Trees are a notation for capturing natural-language requirements in a graphical format in a manner which stays close to the structure and terminology of the original requirements, but with a formal semantics. Geoff Dromey described a method – which he called Behavior Engineering (BE) (Dromey, 2003, 2006) – for developing a Behavior Tree (BT) model from requirements systematically, in such a way that issues with consistency and completeness are revealed and resolved as the tree is built. The resulting tree expresses all the scenarios and use cases that are implied by the requirements in a single coherent model.

Evidence from industry use (Boston, 2008; Powell, 2007) has demonstrated that requirements quality can be significantly improved using this approach, and that the resulting models are much easier for non-experts to understand. This in turn leads to improved requirements understanding early in the system and software development process. The BE method supports

transformation of high-level BTs into more detailed ones in which design decisions are embodied, and there are tools for generating code from sufficiently detailed models, but these aspects of the method are not as widely used. At some point it becomes preferable to switch to more traditional development notations and methods, such as UML and MDE (Schmidt, 2006).

In this paper we define a transformation from basic BT models to UML state machine (SM) models. The full BT notation supports a rich variety of relations, capturing non-functional aspects (the what, why, when, where and how of requirements) as extra annotations to nodes in the Behavior Tree. The stripped-down basic BT notation captures functionality and behaviour, such as control and data flow (Lindsay, 2010) – the “logic” of the requirements. Ensuring the consistency and completeness of this logic can be one of the hardest things for a system developer to get right, and one of the most expensive things to fix if it is wrong (Glass, 2004).

We contend that BT modelling combined with formal transformation to SM models is a highly effective means for going from natural-language system requirements to Model Driven Engineering (MDE) in UML. Conversely, the ability to execute SM models enables dynamic aspects of BT models to be debugged, leading to improved system specifications. We illustrate the approach on a well known case study – the security alarm system from Prowell et al. (1999).

* Corresponding author.

E-mail addresses: Soon-Kyeong.Kim@nicta.com.au (S.-K. Kim),

Toby.Myers@griffithuni.edu.au (T. Myers),

marc-florian.wendland@fokus.fraunhofer.de (M.-F. Wendland),

p.lindsay@uq.edu.au (P.A. Lindsay).

The paper is structured as follows: Section 2 describes related work, including potential applications of the translation. Section 3 describes the basic BT notation and illustrates it on the case study. Section 4 describes the BT-to-SM transformation rules and illustrates them on a set of small examples that capture generic BT structures. Section 5 illustrates how execution of the SM model reveals issues with formulation of requirements in the case study as an application of the transformation. Section 6 summarises our approach and discusses future work. Prior familiarity with UML state machines is assumed (OMG, 2011a).

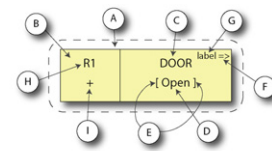
2. Related work

Other papers have presented translational semantics for the BT notation, but ours is the first full translation to state machines. (Grunske et al., 2008) define a graph grammar-based transformation from BT models to Action Systems, as used by the SAL model checking tool. The SAL translation has been used to provide support for Failure Modes and Effects Analysis (Grunske et al., 2011) and Cut Set Analysis (Lindsay et al., 2011) using the BT method. Colvin et al. (2008) extend the BT notation with timing requirements, with a translation to the UPPAAL model checker. They also extend the BT notation with probability and timing, with a translation to the PRISM stochastic model checker in Colvin et al. (2007). Colvin and Hayes (2010) provide a full formal static semantics for the BT notation in an extension of the CSP notation.

Other approaches have been developed for developing SM models from requirements-capture notations, including UML Sequence Diagrams (Whittle and Schumann, 2000), Message Sequence Charts (Kruger, 2000; Uchitel et al., 2003) and Live Sequence Charts (Harel et al., 2005; Meng et al., 2011). As noted in Whittle and Jayaraman (2006), however relationships between scenarios and/or use cases are not defined explicitly, which makes it difficult to generate a complete executable SM. Whittle and Jayaraman (2006) address this problem by explicitly defining relationships between scenarios and use cases using activity diagrams and interaction diagrams, and then generate a SM from these diagrams. By contrast BTs integrate use cases and scenarios into a single model.

Animators and code generators have been developed for BTs, but they are generally still embryonic. By contrast, many simulation tools support UML state machines, to a greater or lesser extent, with varying interpretations of SM semantics. We use the SHIRE tool here (Höfig et al., 2011), with an extension to enable visualisation of execution sequences. Crane and Dingel (2007) present syntactic and semantic differences between the UML SM and its variants (e.g. classical and Rhapsody statecharts) and discuss the implications for supporting tools. OMG has released a new specification notation fUML (OMG, 2011b) within the UML family, that describes a fundamental subset for an executable representation of UML. However, fUML is based solely on activities and as yet does not support the state concept which is vital in mapping BTs to UML, and there is no mature execution tool for fUML as yet.

There are many approaches to model-based testing based on UML state machines. These approaches include generation and specification of test cases (Chevalley and Thévenod-Fosse, 2001; Kim et al., 1999). BT models can be developed directly from high-level functional specifications of systems, so are potentially a strong basis for derivation of system-level test cases, via state machines and SM-based testing tools. The latter include commercial tools (Conformiq, 2011; TestCast MBT, 2011; MBTsuite, 2011) and non-commercial tools (Alin et al., 2010; SpecExplorer, 2011). This topic is the subject on ongoing research.



(A) Behavior Tree node; (B) tag; (C) component name; (D) behaviour note; (E) behaviour type; (F) operator; (G) label; (H) traceability link; (I) traceability status

Fig. 1. Elements of a BT node.

3. Introduction to Behavior Trees

This section describes the BT notation and illustrates it on the security alarm system case study.

3.1. Behavior Tree notation

The Behavior Tree notation is part of a whole methodology of systems and software engineering originally developed by Geoff Dromey called Behavior Engineering (Dromey, 2003). Behavior Trees capture the dynamic behaviour of a system of components in a graphical form. The nodes in the tree describe how components change state in response to flow of data and/or control in multiple parallel threads. Behavior Tree models are developed directly from natural-language system functional requirements by a stepwise process of first translating the behaviour expressed by individual requirements into a partial tree and then integrating the fragments together to form a complete tree. Nodes in the tree are tagged with identifiers of the requirements that gave rise to them, for traceability.

Fig. 1 displays the full contents of a BT node. Each node (A) is associated with a component (C) and has a 'behaviour' (D and E), which is described in more detail below. It can also have an operator (F) and/or label (G), which describe flow of control. The tag (B) has two parts: a link (H) which traces the node back to the requirements that gave rise to it; and a traceability status (I), which the modeller uses to indicate how well the requirement captures the behaviour (not used in this paper).

The different ways nodes can be connected in a Behavior Tree are shown in Fig. 2, together with the different node types and operators. Control flows down branches according to the rules sketched out below: see Behavior Tree Group (2007) for more details. Control flow forks into separate threads when a parallel branching node is reached. For alternative branching, only one thread gets executed, chosen nondeterministically. Although Fig. 2(l–m) show just two branches below a branching node, more than two are also allowed. For the purposes of this paper a fully interleaved control-flow semantics applies. (Sometimes internal actions are given priority over external I/O, but that will not be covered here.) The exception is when nodes are joined by atomic composition (cf. Fig. 2(o)): when flow reaches an atomic grouping, other threads block until all of the nodes in the group have been executed.

The meaning of the different node types is as follows:

- State realisation node `Component1[State1]` indicates that `Component1` is in `State1`.
- Selection node `Component1?Condition1?` is similar to an *if* statement: if `Component1` satisfies `Condition1` when control reaches this node, flow of control continues along the branch; otherwise it terminates. Typically selection nodes appear immediately under an alternative branching node; sometimes a `Component1?ELSE?` is used, to cover the case where all the other selections fail.
- Guard node `Component1???Condition1???` is similar to a *wait* statement: the `Condition1` is continuously re-evaluated and flow

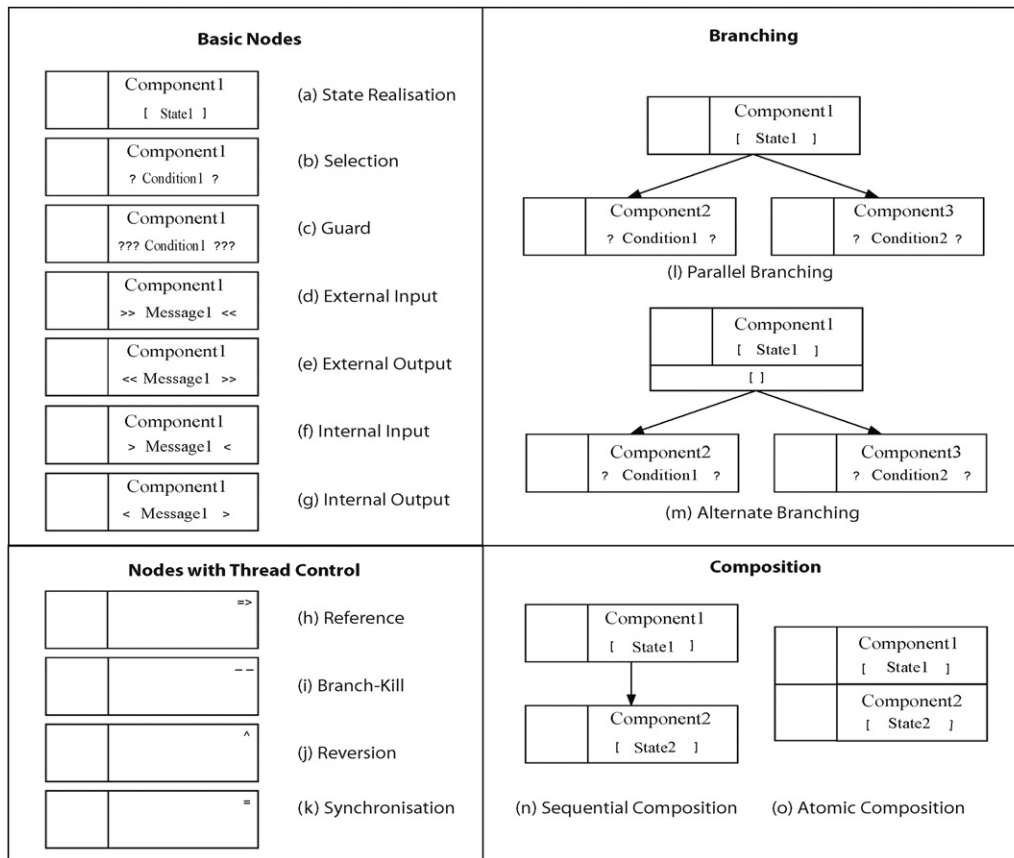


Fig. 2. Summary of the core elements of the BT notation.

of control is blocked at this node until (if ever) *Condition1* becomes *true*.

- Internal output node *Component1* < *Message1* > indicates that *Component1* has broadcast *Message1* on an internal channel, and internal input node *Component1* > *Message1* < indicates that *Component1* has received *Message1* on some internal communication channel. Communication channel details are not modelled in BT notation: they can represent any kind of data flow. Communication is asynchronous. The message is broadcast when control reaches the output node, but flow of control is blocked at an input node until the corresponding output node has been reached, either immediately before the node if it is on the same branch, or in some other thread.
- External input/output is similar to internal I/O, except that external communications channels are involved.

The full BT notation also includes event nodes *Component??Event??* but for this paper's purposes these are indistinguishable from external input nodes so will be omitted.

BT operators (Fig. 2(h–k)) are used to indicate control flow beyond simple sequential execution and branching. In what follows, 'origin node' refers to the node within which the operator appears. The 'target node' is a node elsewhere in the tree which has the same component name and 'behaviour' as the origin node. Exactly which node is the target node depends on the operator involved, as explained below. For the BT model to be well-formed, the target node needs to be unambiguous; see Colvin and Hayes (2010) for the full static semantics of the BT notation. The meaning of the different operators is as follows:

- The reference operator '=' causes control to jump to the target node, which can appear anywhere in tree except at leaf node; reference can only be used at leaf nodes. In essence, reference acts as a macro, equivalent to creating in-place below the origin node a complete copy of the subtree from below the target node.
- The branch kill operator '--' terminates all threads that started at or below the target node. In this case the target node must be a sibling of one of the origin node's ancestors further up the tree.
- The reversion operator '^' causes control to jump back to the target node, which must be one of the origin node's ancestors further up the tree. Reversion causes termination of all threads that started at or below the target node.
- The synchronisation operator '=' causes control flow to block until control reaches the target node. In fact, synchronisation can be applied to multiple threads. Both the origin node and the target nodes get labelled with '='.

If flow of control reaches a leaf node that does not have an operator, the corresponding thread executes the node and terminates.

BTs are defined by a formal semantics defined in CSP σ , an extension of CSP that can capture state based information (Colvin and Hayes, 2010).

3.2. Case study: security alarm system and BT modelling

This section introduces the case study that is used as a running example in this paper – the Security Alarm System. Table 1 shows the seven natural-language requirements that constitute the system functional specification in Prowell et al. (1999).

Table 1
Functional requirements for the Security Alarm System.

R1	The security alarm has a detector that sends a trip signal when motion is detected.
R2	The security alarm is activated by pressing the Set button.
R3	The set button is illuminated when the security alarm is set.
R4	If a trip signal occurs while the security alarm is set, a high-pitched tone (alarm) is emitted.
R5	A three-digit code must be entered to turn off the alarm tone.
R6	Correct entry of the code deactivates the security alarm.
R7	If a mistake is made when entering the code, the user must press the Clear button before the code can be re-entered.

A BT model developed by systematic formulation and integration of the seven requirements is shown in Fig. 3. Nodes are labelled with the requirement(s) that gave rise to them, if any. For example, the first three nodes capture the behaviour of activating the security alarm system. Pressing the Set button is modelled as an external input. This results in the system being activated. (The requirements also use the word 'set' but we stick with 'activated' for consistency in the model.) The rest of the BT model is fairly self-explanatory. External events such as detecting motion, entering a three-digit code, and pressing the Clear button are modelled as external inputs. Passing of the trip signal from detector to the control system is modelled using internal I/O. This BT model is used for validating and improving the requirements in Section 5.

4. Transforming Behavior Trees to UML State Machines

This section describes our transformation rules for mapping BTs to UML state machines, and the transformation environment used in our work.

4.1. Transformation environment and evaluation

Fig. 4 shows the transformation environment used. The mapping from BTs to SMs is based on metamodels of the two languages and implemented using a model-to-model (M2M) transformation defined in the ATLAS Transformation Language (ATL) (ATL, 2011) based on EMF. Fig. 5 shows the BT metamodel used in the EMF-based TextBE tool (Myers, 2011). The standard OMG UML Superstructure (version 2.2) (OMG, 2011a) was used for UML, with an EMF-based implementation which is developed as part of the Eclipse Model Development Tools (MDT) project (UML2Tools, 2011). The described transformation has been included as part of the TextBE tool. Visualisation was performed with Eclipse UML2Tools (UML2Tools, 2011).

A number of examples were transformed and simulated to validate the SM conformed to the semantics of the associated BT. SHIRE (Höfig et al., 2011) was used to simulate the generated SM models. We also applied the transformation rules to several large case studies (e.g., the Microwave Oven system, the Security Alarm System and the Automatic Train Station system). Simulation proved to be a useful means of validating the rules and the BT models. Feedback gained from the simulation were used to enhance the transformation rules and the BT models. Due to the complexity of some of the transformation rules, not all of them were implemented fully in ATL. Nevertheless, the SM models presented in this paper are all generated by using our ATL implementation of the transformation rules. The BT models and the generated SM models used for the evaluation are available from Myers and Kim (2012).

4.2. The transformation rules

The transformation maps a BT into a single monolithic state machine. Naming conventions and a variable associated with each

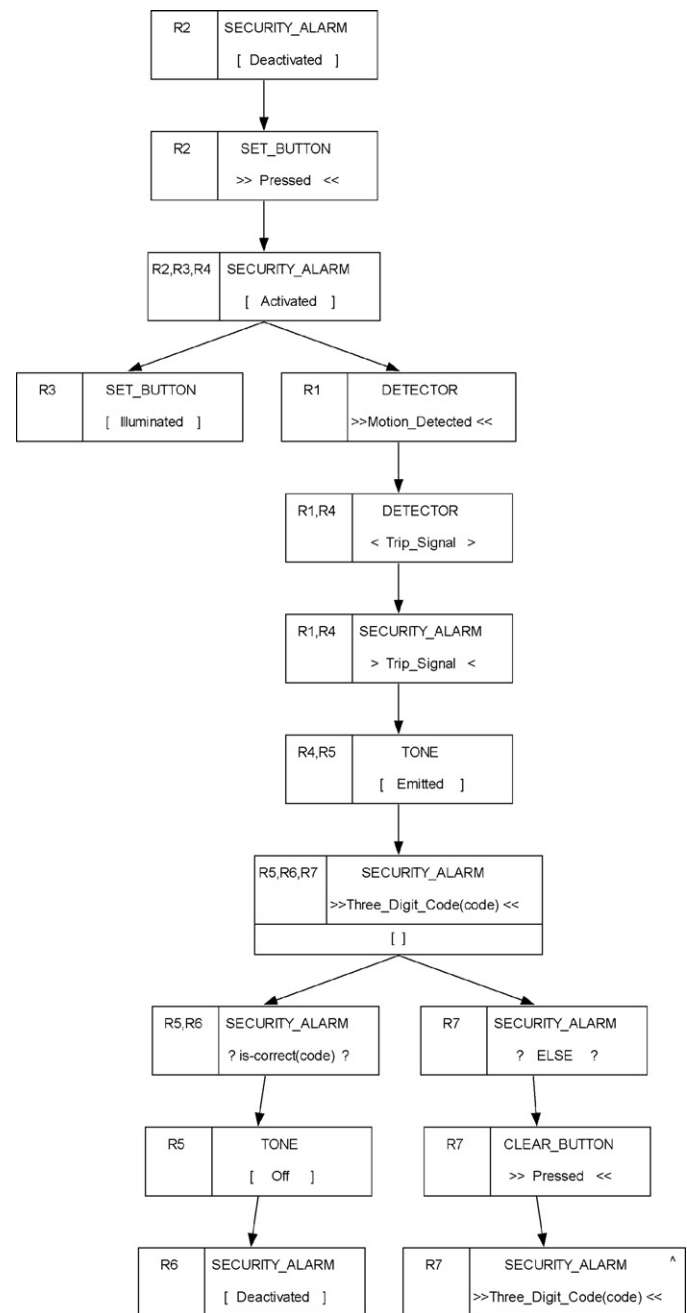


Fig. 3. Initial BT model of the Security Alarm System.

component are used to preserve component behaviour in the SM. The transformation supports all the core elements of the BT notation shown in Fig. 2. In what follows the transformation rules for each BT construct are explained and illustrated using example BT models. Familiarity with SM concepts is assumed, such as entry actions, completion events, signals, and so on: see OMG (2011a) for details.

4.2.1. State realisation nodes

A BT state realisation node maps to an SM state, although different instances of a node typically are given different names in the SM, for reasons explained below. To make the mapping easier to understand, a naming convention is used whereby the SM state's name is composed of the BT component's name and (BT) state. A UML classifier attribute is introduced to track the component's state: see Fig. 6 for an example. The attribute gets set to the

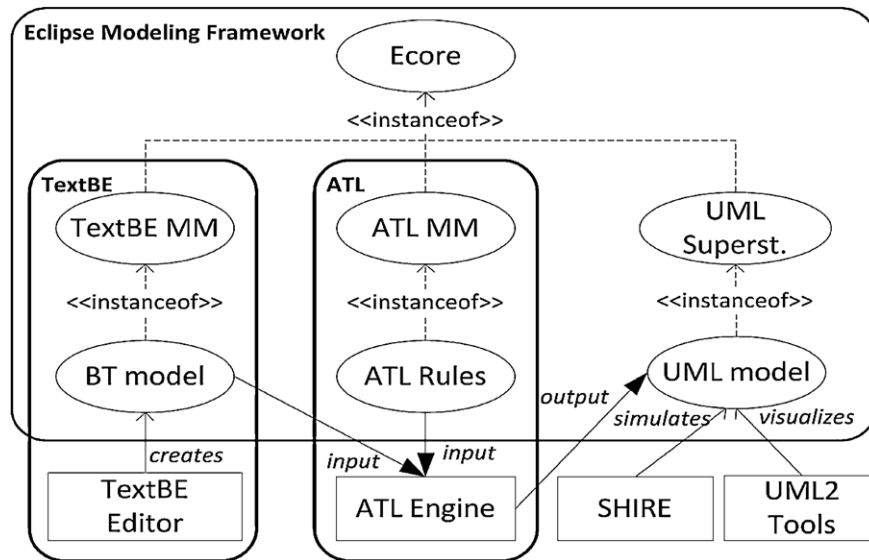


Fig. 4. Transformation environment.

appropriate value using an entry action in the SM. This attribute will be de-referenced in SM transitions corresponding to selection and guard nodes, for example.

Duplicate BT state realisation nodes are distinguished in the SM by appending a unique numeral to the name. This is necessary because BT models describe flow of control, not simply state changes: the position of the node in the tree describes the system behaviours involving the component, which can be very different. For example, in the BT fragment shown in Fig. 6(a), the first and last nodes are identical but no further transition is possible from the last node and it is connected to a final state.

As in this example, when BT state realisation nodes are sequentially composed, we could connect the SM states with a completion transition. However, this translation raises an issue when the sequentially composed nodes are defined in BT parallel branches. The SM semantics that prioritise completion transitions differs

from the BT semantics that allow interleaving between sequentially composed nodes in parallel branches during execution. This will not be able to occur in the transformed SM if the sequentially composed nodes are connected by a completion transition. To address this issue, we exclude SM composition transitions from our translation. Instead, we create an event called 'comp' and use this event to trigger transitions between the SM states transforming the sequentially connected BT nodes. In this way, we preserve the interleaving semantics of BT in the SM model.

4.2.2. Input/output nodes, parallel branching and reversion operator

Parallel BT branching translates to an orthogonal composite state with a region for each branch in the SM model. BT input nodes translate to SM transitions with triggers, and BT output nodes translate to SM transitions with effects. The message in the BT

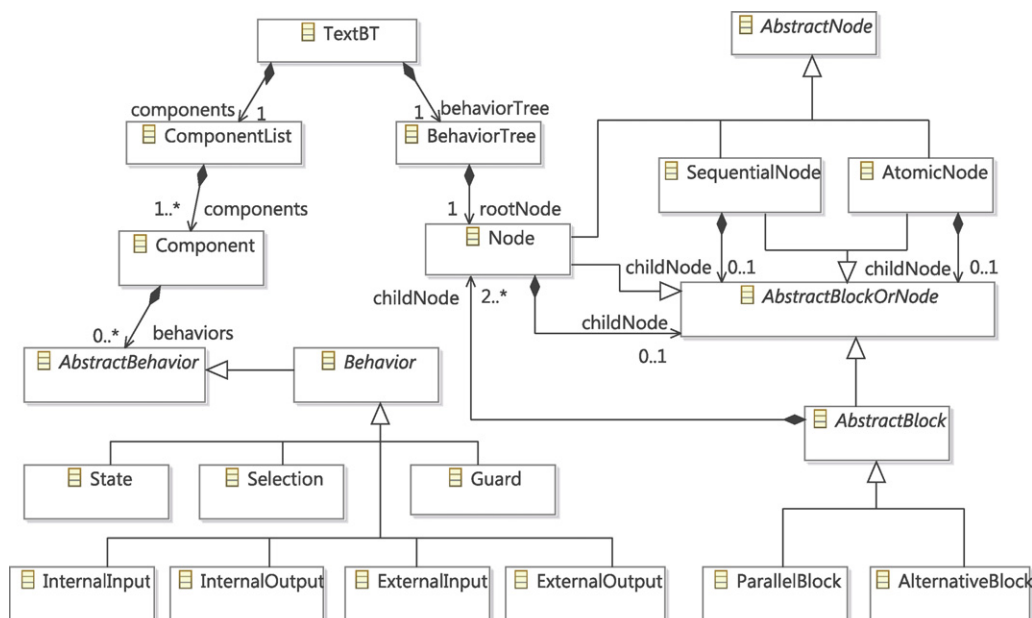


Fig. 5. The BT metamodel used in TextBE.

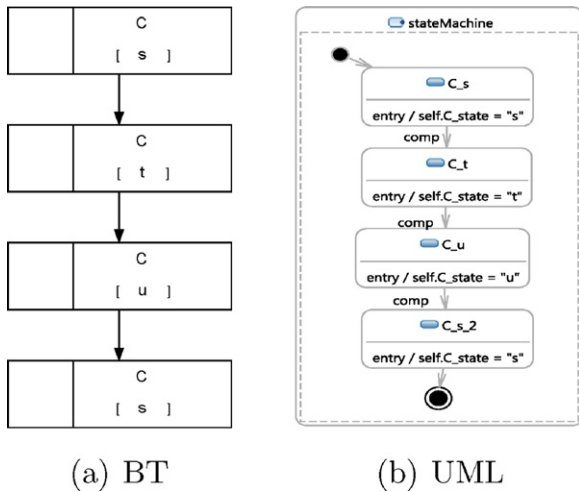


Fig. 6. Example of translation of BT state realisation and input.

node is translated to a signal event in both cases. If the message is parameterised with a data value, then a property is also created for the event and an effect is added to the transition that assigns the parameter value to the property. The property is used in the guards associated with the parameter value in the SM. In our SHIRE-based prototype, code `comms.raise("xxx")` is used to generate the signal event corresponding to the BT output node with message `xxx`, and code `yyy = inputEvent.properties('yyy')` is used to assign a parameter value to a property `yyy`. Intermediate (we call them *dummy*) states are added to the SM model to allow for intervening transitions elsewhere in the BT model: see Fig. 7 for an example of input and output nodes.

Internal and external messages are not distinguished in our translation since the SM model is not concerned with system boundary issues. (In some treatments of BT semantics, internal transitions get priority over external transitions but such matters will not be treated here.)

Recall that the BT reversion operator causes control to flow back to the node with the same details above the current node and kills any threads that were created below that node. To mimic this in the

SM model is relatively straightforward: We create a transition from the current SM state back to the appropriate state, which may be one or more levels outside the current state. This transition results in the composite states corresponding to the other threads (if any) being killed, as desired. Fig. 7 shows an example.

4.2.3. Selection nodes, guard nodes and alternative branching

As noted above, BT selection nodes correspond to *if* statements, with termination if their condition is not met. To mimic this in the SM model, for each selection node a transition is introduced that leads to (the translation of) the tree below the node; the transition is labelled with a guard corresponding to the condition from the BT selection node. A transition to a final state is also added, for the case where none of the conditions hold; it is labelled with a guard corresponding to the negation of the conditions of all of the selection nodes.

Often BT selection nodes occur at the head of alternative branches, sometimes using an *?ELSE?* node to capture the “otherwise” case (where none of the conditions in the other branches are true): see Fig. 8(a) for an example. In this case, instead of being a final state, a transition is created for (the translation of) the tree below the ELSE node. Fig. 8(b) shows an example.

A BT guard node is used to suspend behaviour until some parallel thread results in the node's component realising the named state. This semantics is analogous to UML change events as a change event's expression can relate to the values of parallel threads of behaviour. Based on this semantic comparability, we could simply map a BT guard node to a transition with a change event as its trigger. However, change events are not supported by the simulation tools that we investigated (e.g. SHIRE, SCXML, Topcased and Rhapsody). To mimic the behaviour of a BT guard node in the SM model without using change events, we add a dummy state corresponding to suspended behaviour and two transitions. The first transition targets (the translation of) the behaviour below the BT node and has an SM guard checking whether the component is in the desired state (see Section 4.2.1 above for details of the UML attribute involved). The second transition loops back to the dummy state and has a guard which is the negation of the one just explained; this enables continuous re-evaluation of the component state. Fig. 10 for an example.

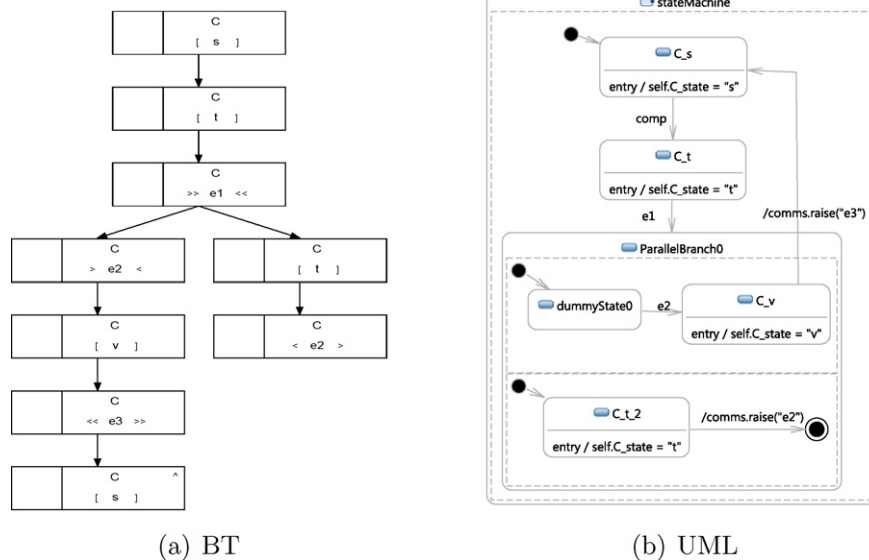
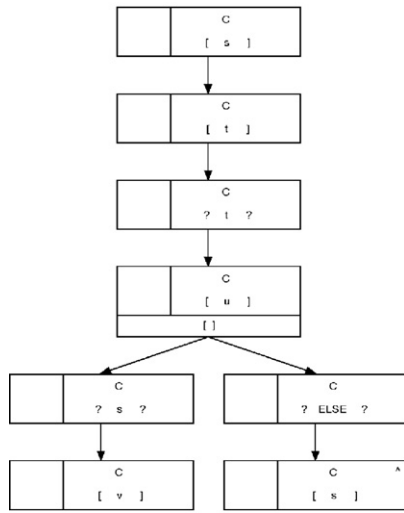
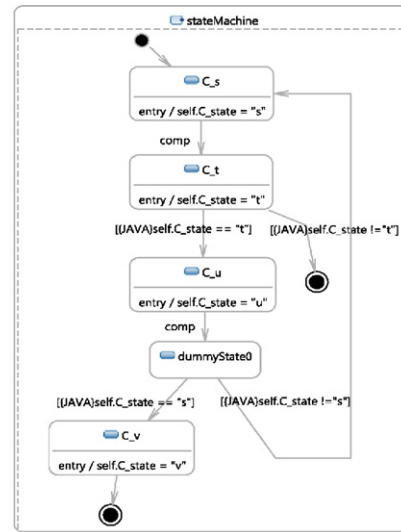


Fig. 7. Example of translation of BT I/O, parallel branching and reversion operator.

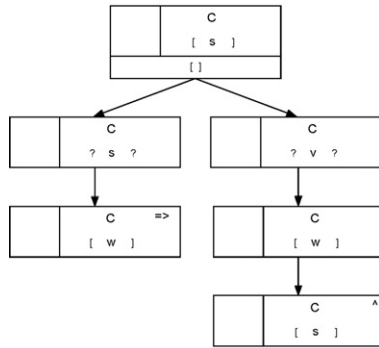


(a) BT

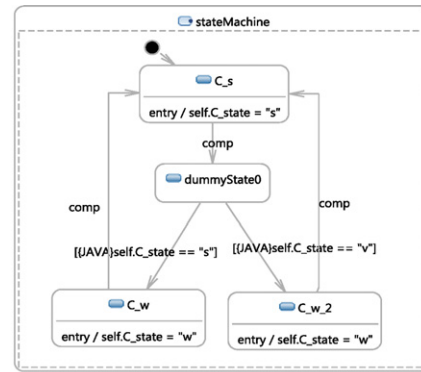


(b) UML

Fig. 8. Example of translation of BT selection and alternative branching.

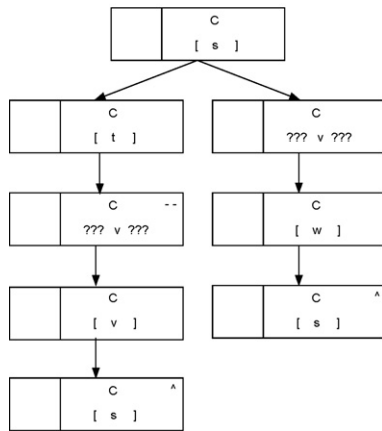


(a) BT

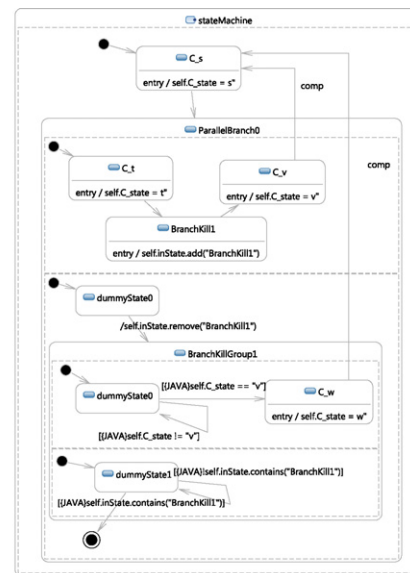


(b) UML

Fig. 9. Example of translation of BT reference.

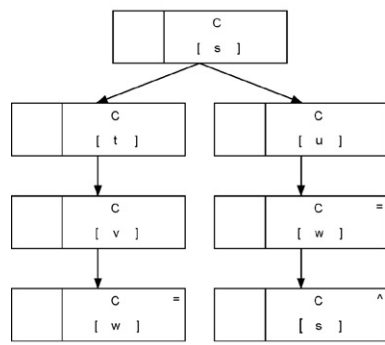


(a) BT

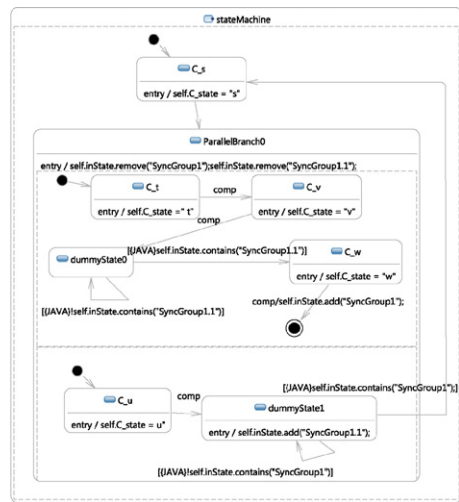


(b) UML

Fig. 10. Example of translation of BT branch kill node.

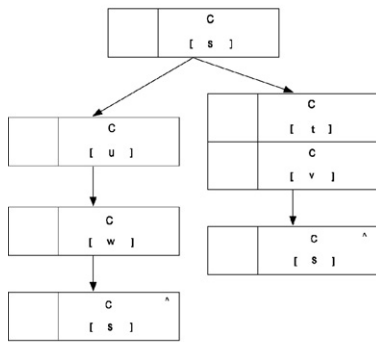


(a) BT

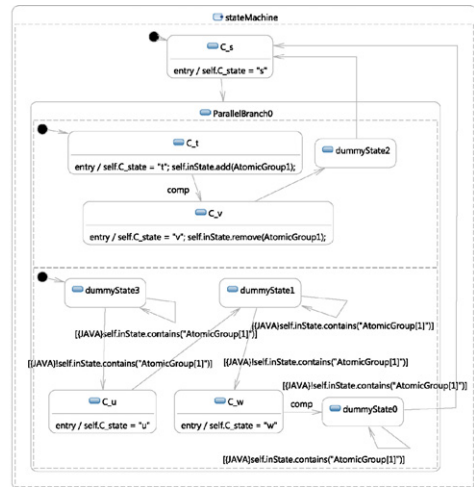


(b) UML

Fig. 11. Example of translation of BT synchronisation nodes.



(a) BT



(b) UML

Fig. 12. Example of translation of BT atomic composition.

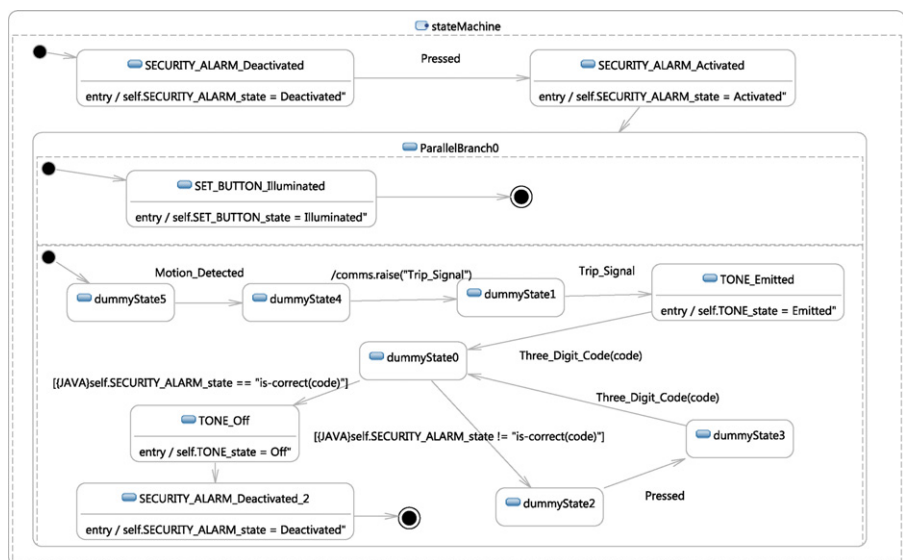


Fig. 13. SM model generated from the BT model in Fig. 3.

4.2.4. Reference operator

Recall that the BT reference operator acts simply like a jump in control flow. In many cases it thus simply gives rise to a transition from the current SM state to the appropriate target state. Fig. 9 shows an example. The difficulty arises if composite states intervene since, unlike the reversion operator, the reference operator does not kill other threads. In these cases we instead treat reference as a kind of ‘macro’ and create a complete copy of SM corresponding to the tree below the target node.

4.2.5. Branch kill and synchronisation operators

Translation of branch kill nodes is more tricky since it involves going back up the tree to the node where the to-be-killed thread branches off, and introducing some more structure into the SM model from this point onwards. An attribute ‘*inState*’ is added to the SM which keeps track of all threads that might need to be killed (as identified by kill nodes in the BT model). At each point in the SM model where such a thread gets started we insert a composite state composed of two substates. The first of the two sub-states corresponds to that part of the SM which results from translation of the corresponding BT subtree. The other sub-state handles killing of the thread by having a transition to a final state at the outer level, which terminates the first sub-state when the time comes. This second sub-state continuously checks to see if the thread has been added to the *inState* attribute. Finally, the branch kill node itself gets translated to a state with an entry action which adds the thread to *inState*. Also, the transition into the composite state has an effect that removes the branch kill identifier from *inState*, in case it has previously been added. Fig. 10 shows an example.

Translation of synchronisation nodes is also handled using the *inState* attribute which keeps track of all instances of synchronisation nodes to check whether they have reached the synchronisation point. A dummy state is inserted before (the translation of) the origin synchronisation node with two outgoing transitions. Target synchronisation nodes are translated to a dummy state with two outgoing transitions. The incoming transition to the dummy states corresponding to the translation of the target nodes has an effect that adds a unique synchronisation identifier for that node to *inState*. The outgoing transitions of the dummy state inserted for the origin node have a guard which checks whether all instances of the target nodes have been added to *inState*. If it does, then the transformation of the synchronisation node is executed. Then another identifier is added to *inState* that indicates the synchronisation has occurred. The outgoing transitions of the dummy state of all the target nodes have guards that continuously check for the synchronisation completion identifier (set by the origin node). When included in *inState*, the flow of control continues. Also, the transition into the composite state has an effect that removes the synchronisation identifiers from *inState*, in case they have previously been added. Fig. 11 shows an example.

4.2.6. Atomic composition

The final BT construct to be explained is atomic composition. The translation goal is straightforward enough: we need to block all other SM transitions until the transitions for the atomic nodes have taken effect. This is achieved using a kind of semaphore mechanism similar to that used for synchronisation. An attribute is introduced for each atomic group. When control reaches such a group, the attribute gets set to *True* until (BT) control flow leaves the group. Dummy states are inserted between transitions in all other sections of the SM model, with self-transitions when the attribute is true and progress to the next state when false. Fig. 12 shows an example.

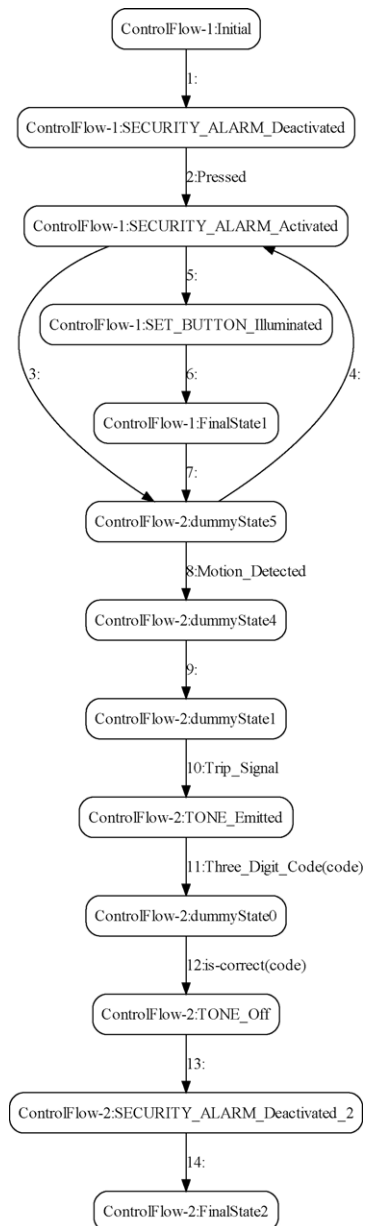


Fig. 14. Example simulation run for the SM model for Fig. 13.

The SM model in Fig. 13 was generated by applying the transformation rules to the BT model in Fig. 3. This is used for validation in the following section.

5. Requirements analysis and improvement by SM execution

This section illustrates how execution of the SM model can be used to improve understanding and formulation of the case study requirements from Section 3.2. We start with the SM model in Fig. 13, which resulted from transformation of the BT model of the requirements in Fig. 3. At each step we use the SHIRE simulation environment to explore scenarios, to check that the system behaves as desired. We extended SHIRE with graphical output, to better visualise simulation results. When we encounter unexpected or undesired behaviour we modify the BT model to correct the behaviour and rerun the scenario until the desired behaviour is attained. We then rewrite the requirements to reflect our improved understanding.

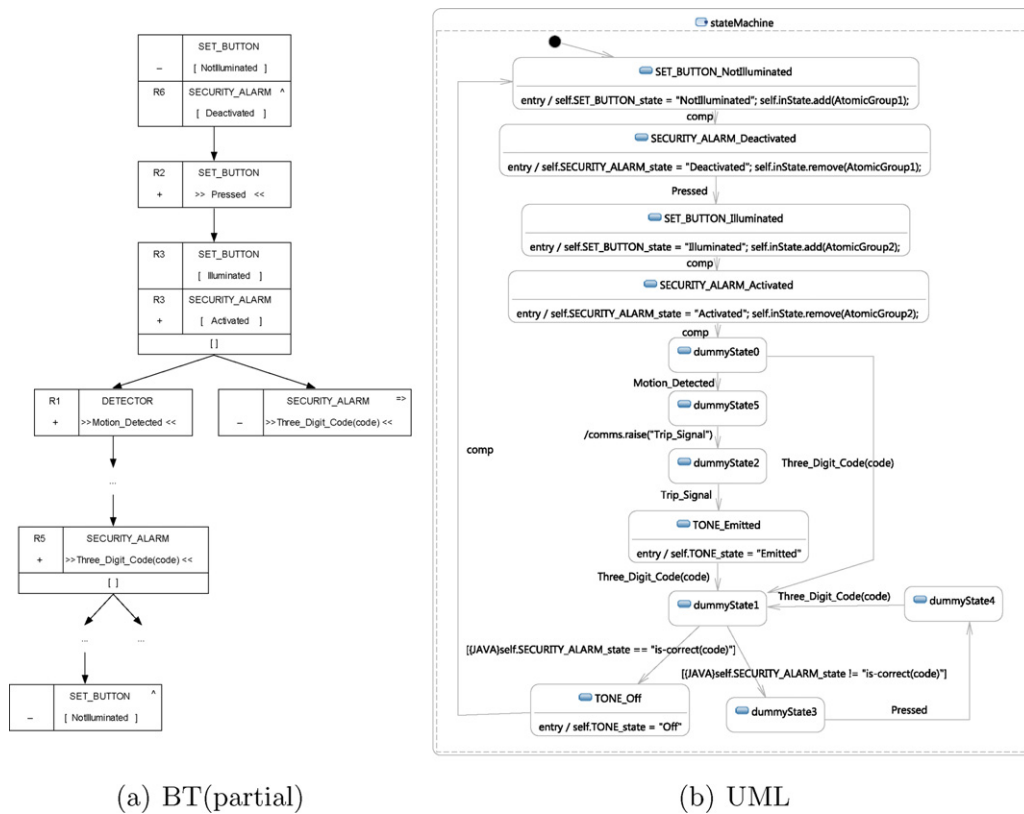


Fig. 15. The revised BT and UML models.

The rest of this section describes four stepwise modifications of the requirements resulting from the above approach. Each step is described in four parts: validation goal (the expected behaviour of the system); simulation results (one or more scenarios where execution either failed or gave unexpected behaviour); correction (modification of the BT model and successful rerun of the scenario); and rewording of the offending requirements.

5.1. Button illumination

Validation goal: SET_BUTTON should be illuminated when the security alarm is activated, so users can see that the system is active.

Simulation results: After the user press-button event it was possible to activate the system without the button actually getting illuminated. Fig. 14 shows one such scenario (see transitions 1–4).

Correction: The Set button should be illuminated at the same time as the rest of the system is activated, rather than as a parallel behaviour.

Reworded requirement: The following revision of R3 reflects our improved understanding: “The Set button is illuminated whenever the security alarm is activated.”

5.2. Deactivating the system

Validation goal: Closer inspection of the SM model reveals that if the button illuminates then it remains illuminated thereafter. But the desired behaviour is that the button should cease to be illuminated when the system is deactivated.

Simulation results: We ran a scenario in which the system got deactivated by the user entering a correct code. But at that point the SM terminated, with the button still illuminated (see the transitions 6–14 in Fig. 14). Here we treat the termination issue first and then

return to the button illumination issue in a separate experiment below.

Correction: The simulation result shows the last steps of the run that terminated. There should be a reversion back to the deactivated state from the final BT node, so the system can keep operating.

Reworded requirement: R6 should be clarified to say “Correct entry of the three-digit code deactivates the security alarm and returns the security alarm to its deactivated state.”

5.3. De-illuminating the button

Validation goal: The button should de-illuminate once the system is deactivated, so the user can see that the system is no longer active.

Simulation results: In the previous experiment, the light remained illuminated after the system was deactivated.

Correction: The button needs to get de-illuminated when the system gets deactivated. The best way to achieve this is to insert a set button de-illuminated node at the beginning of the BT model and compose this node with the security alarm deactivated node atomically, and insert a reversion node to this atomic block.

Reworded requirement: Add a clarifying requirement: “R8. The Set button is not illuminated whenever the security alarm is deactivated.”

5.4. Deactivating the system (II)

Validation goal: The user should be able to deactivate the system without needing to trip the alarm.

Simulation results: When executing the current SM model it is apparent that the only way to reach the system deactivated state is to first detect motion (see transition 8 in Fig. 14). Yet clearly deactivation needs to be possible independent of whether motion is detected or not.

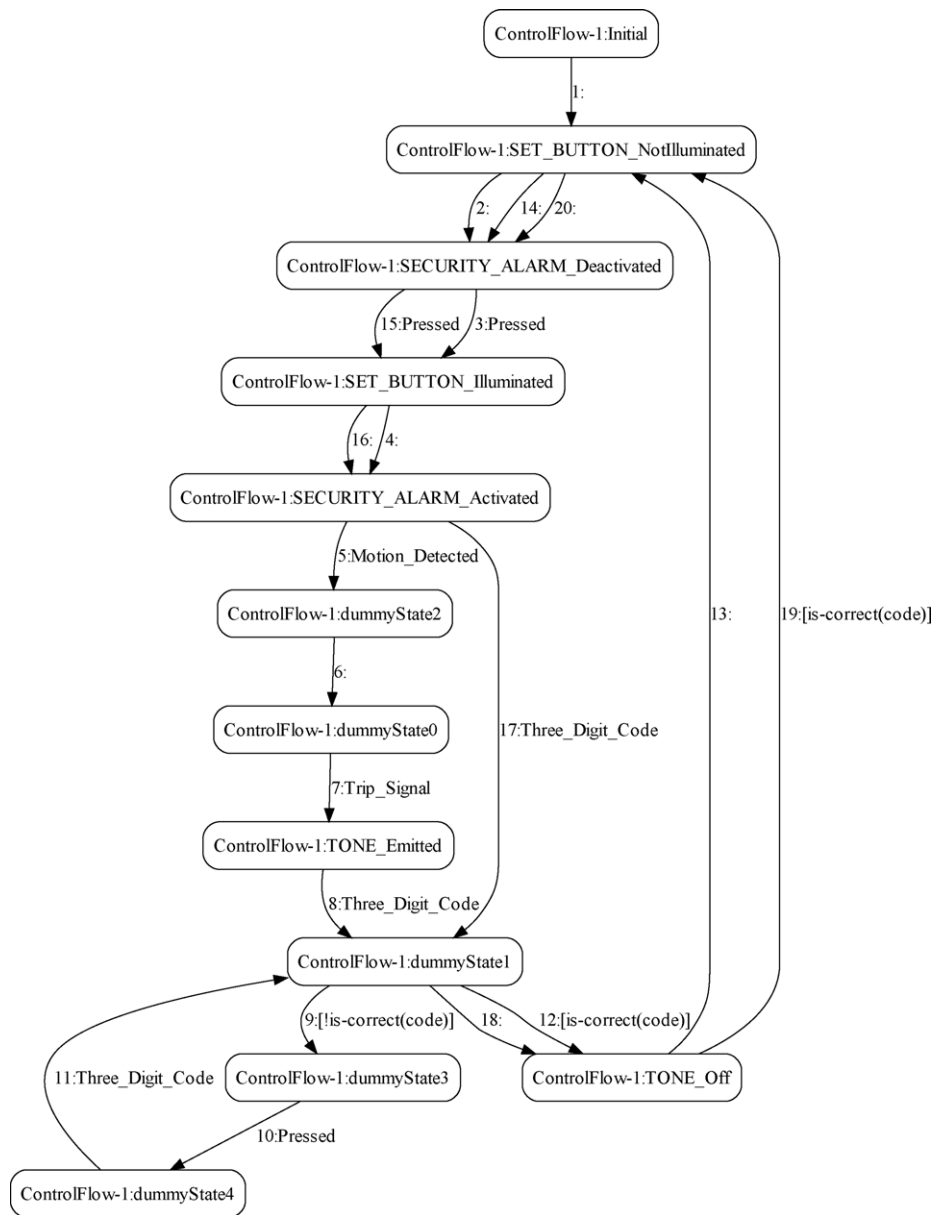


Fig. 16. Example simulation run for the SM model for Fig. 15(b).

Correction: We fixed this issue by inserting a new alternative branch under the SECURITY_ALARM[Activated] node, with a reference to the code-entry subtree.

Reworded requirement: R5 should be clarified to say “A three-digit code must be entered to deactivate the security alarm.” and a new requirement added: “R9. To turn off the alarm tone, the user must deactivate the security alarm by entering a correct three-digit code.”

5.5. Improved requirements and models

Fig. 15(a) shows the BT model with the corrections made. Fig. 15(b) shows the resulting SM model. The simulation result in Fig. 16 shows two successful scenarios of the system behaviour where the user can de-illuminate the button and deactivate the system by entering correct code (transitions 5–13), even if either without motion being detected (transitions 16–19).

The final set of requirements is given in Table 2 below. Three requirements (R3, R5 and R6) are reworded and two clarifying

Table 2

Reworded requirements for the Security Alarm System.

R1	The security alarm has a detector that sends a trip signal when motion is detected.
R2	The security alarm is activated by pressing the Set button.
R3	The Set button is illuminated whenever the security alarm is activated.
R4	If a trip signal occurs while the security alarm is activated, a high-pitched tone (alarm) is emitted.
R5	A three-digit code must be entered to deactivate the security alarm.
R6	Correct entry of the three-digit code deactivates the security alarm and returns the security alarm to its deactivated state.
R7	If a mistake is made when entering the code, the user must press the Clear button before the code can be re-entered.
R8	The Set button is not illuminated whenever the security alarm is deactivated.
R9	To turn off the alarm tone, the user must deactivate the security alarm by entering a correct three-digit code.

requirements (R8 and R9) are newly added. 'Set' has been changed to 'activated' for consistency.

6. Conclusion and future work

In summary, the main contribution of this paper is to provide a detailed transformation of models in the BT notation to UML state machines (SMs). The BT notation was developed to improve the modelling and analysis of system requirements expressed in natural language, staying as close as possible to the original natural-language description, to better preserve the intent of the original requirements (Dromey, 2005). We demonstrated how transformation of BT models to SMs and use of SM execution tools enables validation and improvement of requirements. This idea can be taken further, to provide a rigorous path from natural language requirements specifications to Model Driven Engineering. The transformation also provides a formalisation of the semantics of dynamic behaviour represented by BT notation, alternative to the CSP σ formalisation given in Colvin and Hayes (2010).

The BE methodology also supports the systematic development of detailed designs from requirements BTs (Myers, 2010). Our transformation works equally well for Requirements BTs and Design BTs: the point at which one switches from the BE methodology to Model Driven Engineering in UML is the developer's decision. BTs provide a coherent and complete framework for expressing scenarios and use cases in a single integrated model. The approach was illustrated on the well-known security alarm system example (Prowell et al., 1999), using the TextBE implementation of BTs and the SHIRE implementation of executable state machines which both operate on EMF-based models. We were able to use the SM model to animate behaviour of the security alarm system and validate the requirements and the BT model. We have applied our work to several other case studies including a complex system, Automatic Train Station (BE, 2011) and have validated the system via simulation. In future work we plan to exploit SM tool capabilities to extend the range of software analysis and development techniques that can operate on requirements specifications, such as automated generation of test cases. We also plan to apply our work to industrial applications such as Air Traffic Management systems.

Acknowledgements

NICTA is funded by the Australian Government as represented by the Department of Broadband, Communications and the Digital Economy and the Australian Research Council through the ICT Centres of Excellence program. This research was also supported under Australian Research Council *Linkage Projects* funding scheme (project number LP0989363) and the BTTest project, a joint research project of University of Queensland (Australia) and Fraunhofer FOKUS (Germany), funded on German side by the DAAD (project number 50449190). We also would like to thank Dr. Edzard Höfig for providing us a free research license of SHIRE to accomplish this work.

References

- ATL Transformation Website, 2011. <http://www.eclipse.org/atf/>.
- Alin, S., Sebastian, W., Marc-Florian, W., 2010. Using the UML testing profile for enterprise service choreographies. In: 36th Conference on Software Engineering and Advanced Applications, pp. 12–19.
- BE, 2011. Behaviour Engineering: Case Studies., http://www.behaviorengineering.org/index.php?option=com_content&task=view&id=14&Itemid=28.
- Behavior Tree Group, 2007. Behavior Tree Notation., <http://www.behaviorengineering.org/docs/Behavior-Tree-Notation-1.0.pdf>.
- Boston, J., 2008. Behavior trees – how they improve engineering behaviour? In: 6th Annual Software and Systems Engineering Process Group Conference (SEPG 2008), Melbourne, Australia.
- Chevalley, P., Thévenod-Fosse, P., 2001. Automated generation of statistical test cases from UML state diagrams. In: COMPSAC. IEEE Computer Society, pp. 205–214.
- Colvin, R., Grunske, L., Winter, K., 2007. Probabilistic timed Behavior Trees. In: Proceedings of the International Conference on Integrated Formal Methods (IFM), LNCS, vol. 4591. Springer-Verlag, pp. 156–175.
- Colvin, R., Grunske, L., Winter, K., 2008. Timed behavior trees for failure mode and effects analysis of time-critical systems. *Journal of Systems and Software* 81, 2163–2182.
- Colvin, R.J., Hayes, I.J., 2010. A semantics for Behavior Trees using CSP with specification commands. *Science of Computer Programming* 76, 891–914.
- Conformiq, 2011. <http://www.conformiq.com/>.
- Crane, M.L., Dingel, J., 2007. UML vs. classical vs. rhapsody statecharts: not all models are created equal. *Software and System Modeling* 6, 415–435.
- Dromey, R.G., 2005. Genetic design: amplifying our ability to deal with requirements complexity. In: Scenarios: Models, Transformations and Tools. Springer-Verlag, pp. 95–108.
- Dromey, R.G., 2003. From requirements to design: formalizing the key steps. In: International Conference on Software Engineering and Formal Methods (SEFM 2003). IEEE Computer Society, pp. 2–13.
- Dromey, R.G., 2006. Climbing over the 'no silver bullet' brick wall. *IEEE Software* 23, 118–120.
- Glass, R.L., 2004. Is this a revolutionary idea, or not? *Communications of the ACM* 47, 23–25.
- Grunske, L., Winter, K., Yatapanage, N., 2008. Defining the abstract syntax of visual languages with advanced graph grammars – a case study based on behavior trees. *Journal of Visual Languages and Computing* 19, 343–379.
- Grunske, L., Winter, K., Yatapanage, N., Zafar, S., Lindsay, P.A., 2011. Experience with fault injection experiments for FMEA. *Software: Practice and Experience* 41, 1233–1258.
- Harel, D., Kugler, H., Pnueli, A., 2005. Synthesis revisited: generating statechart models from scenario-based requirements. In: Formal Methods in Software and Systems Modeling, LNCS, vol. 3393. Springer-Verlag, pp. 309–324.
- Höfig, E., Deussen, P., Schieferdecker, I., 2011. On the performance of UML state machine interpretation at runtime. In: 6th International Symposium on Software Engineering for Adaptive and self-managing Systems. ACM Press, pp. 118–127.
- Kim, Y., Hong, H., Bae, D., Cha, S., 1999. Test cases generation from UML state diagrams. In: IEE Proceedings Software. IEE, pp. 187–192.
- Kruger, I., 2000. Distributed System Design with Mess Sequence Charts. Ph.D. Thesis. Technical University Munich.
- Lindsay, P.A., 2010. Behavior Trees: From Systems Engineering to Software Engineering. In: 8th IEEE International Conference on Software Engineering and Formal Methods (SEFM), IEEE Computer Society, pp. 21–30.
- Lindsay, P.A., Yatapanage, N., Winter, K., 2011. Cut Set Analysis using Behavior Trees and model checking. *Formal Aspects of Computing*, <http://dx.doi.org/10.1007/s00165-011-0181-8>.
- MBTSuite, 2011. <http://www.seppmed.de/produkte/mbtsuite.html>.
- Meng, S., Arbab, F., Baier, C., 2011. Synthesis of Reo circuits from scenario-based interaction specifications. *Science of Computer Programming* 76, 651–680.
- Myers, T., 2010. The foundations for a scaleable methodology for systems design. Ph.D. Thesis. School of Computer and Information Technology, Griffith University, Australia.
- Myers, T., 2011. TextBE: a textual editor for behavior engineering. In: Proceedings of the 3rd Improving Systems and Software Engineering Conference (ISSEC).
- Myers, T., Kim, S.K., 2012. Transformation Examples of BT to UML State Machines., <http://www.behaviorengineering.org/BT2SMEExamples.pdf>.
- OMG, 2011a. OMG's UML Website., <http://www.uml.org/>.
- OMG, 2011b. Semantics of a Foundational Subset for Executable UML Models (FUML), <http://www.omg.org/spec/FUML/>.
- Powell, D., 2007. Requirements evaluation using Behavior Trees – findings from industry. In: Industry Track of Australian Software Engineering Conference (ASWEC 2007).
- Prowell, S., Trammell, C., Linger, R., Poore, J., 1999. Cleanroom Software Engineering: Technology and Process. Addison-Wesley-Longman.
- Schmidt, D.C., 2006. Guest editor's introduction: model-driven engineering. *IEEE Computer* 39, 25–31.
- SpecExplorer, 2011. <http://research.microsoft.com/en-us/projects/specexplorer/>.
- TestCast MBT, 2011. <http://www.elvior.com/mbt-2/mbt>.
- Uchitel, S., Kramer, J., Magee, J., 2003. Synthesis of behavioral models from scenarios. *IEEE Transactions on Software Engineering* 29, 99–115.
- UML2Tools, 2011. EMF Model Development Tools (MDT) Website., <http://www.eclipse.org/modeling/mdt/?project=uml2tools>.
- Whittle, J., Jayaraman, P.K., 2006. Generating hierarchical state machines from use case charts. In: 14th IEEE International Requirements Engineering Conference. IEEE Computer Society, pp. 16–25.
- Whittle, J., Schumann, J., 2000. Generating statechart designs from scenarios. In: 22nd IEEE International Conference on Software Engineering (2000), pp. 314–323.

Dr Soon-Kyeong Kim is a senior researcher in the Software Systems Research Group at National ICT Australia. She received a PhD in Software Engineering from the University of Queensland in Australia. Previously she has held academic and research positions at UQ. Her research concerns how to apply rigorous modeling techniques to improve software engineering practice, particularly in areas like large complex systems and safety critical systems. She also has strong interests in

model-driven software engineering, method integration, and formalising modeling language semantics and design pattern concepts.

Toby Myers is an adjunct research fellow in the School of Information and Communication Technology at Griffith University, Brisbane, Australia. His research interests are in the continuing development and application of behaviour engineering methodology to the engineering and re-engineering of large-scale integrated systems. He received a PhD in software engineering from Griffith University.

Marc-Florian Wendland is a researcher at the Fraunhofer Institute FOKUS in Berlin, Germany. He works in the competence center MOTION – Modelling and Testing, as an expert for model-driven/based solutions, particularly in the area of quality assurance methods. He is responsible for the development of Fokus!MBT, a tool for MBT approaches based on the well-known standards UML, UTP, SysML and

TTCN-3 and a UTP-based modeling methodology for testing. As OMG member, he leads the revision of the UML Testing Profile and is further involved in standardization activities for UML, SysML and ReqIF.

Dr Peter Lindsay is Boeing Professor of Systems Engineering at The University of Queensland. He has held academic and research positions at the University of NSW, the University of Manchester and the University of Illinois at Urbana-Champaign. He is co-author of two books on formal specification and verification of software systems. He has worked on safety and security-critical applications in areas such as air traffic control, embedded medical devices, ship-board defence, emergency service dispatch systems, and an international diplomatic network. His research interests include engineering of complex systems, safety-critical systems, air traffic management, and formal methods of system development.